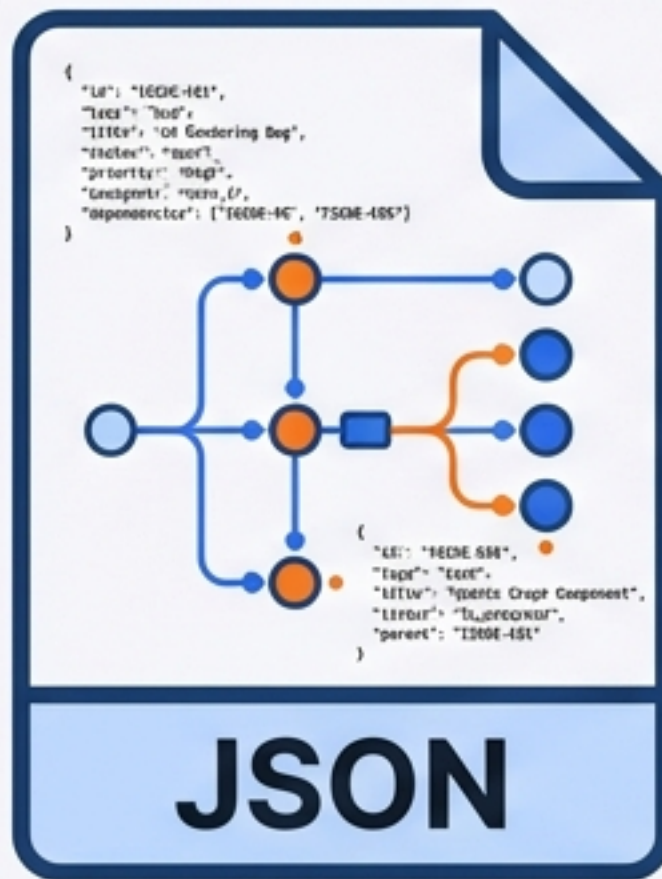


v0.1.3: Issue Editor & Graph Visualisation

A Git-Native, AI-Readable Approach to Project Management



This brief outlines the scope for the next iteration of our git-native issue tracking system. The focus is on bridging the gap between human planning and AI execution through structured data.

The Problem of Invisible Context

Traditional issue trackers (Jira, Linear) live outside your codebase, making them invisible to AI assistants like Claude Code.

Traditional Tracker



- Siloed Data: Issues stored in separate DB.
- Version Drift: Not synced with code.
- Manual Handoff: Requires copy-paste.

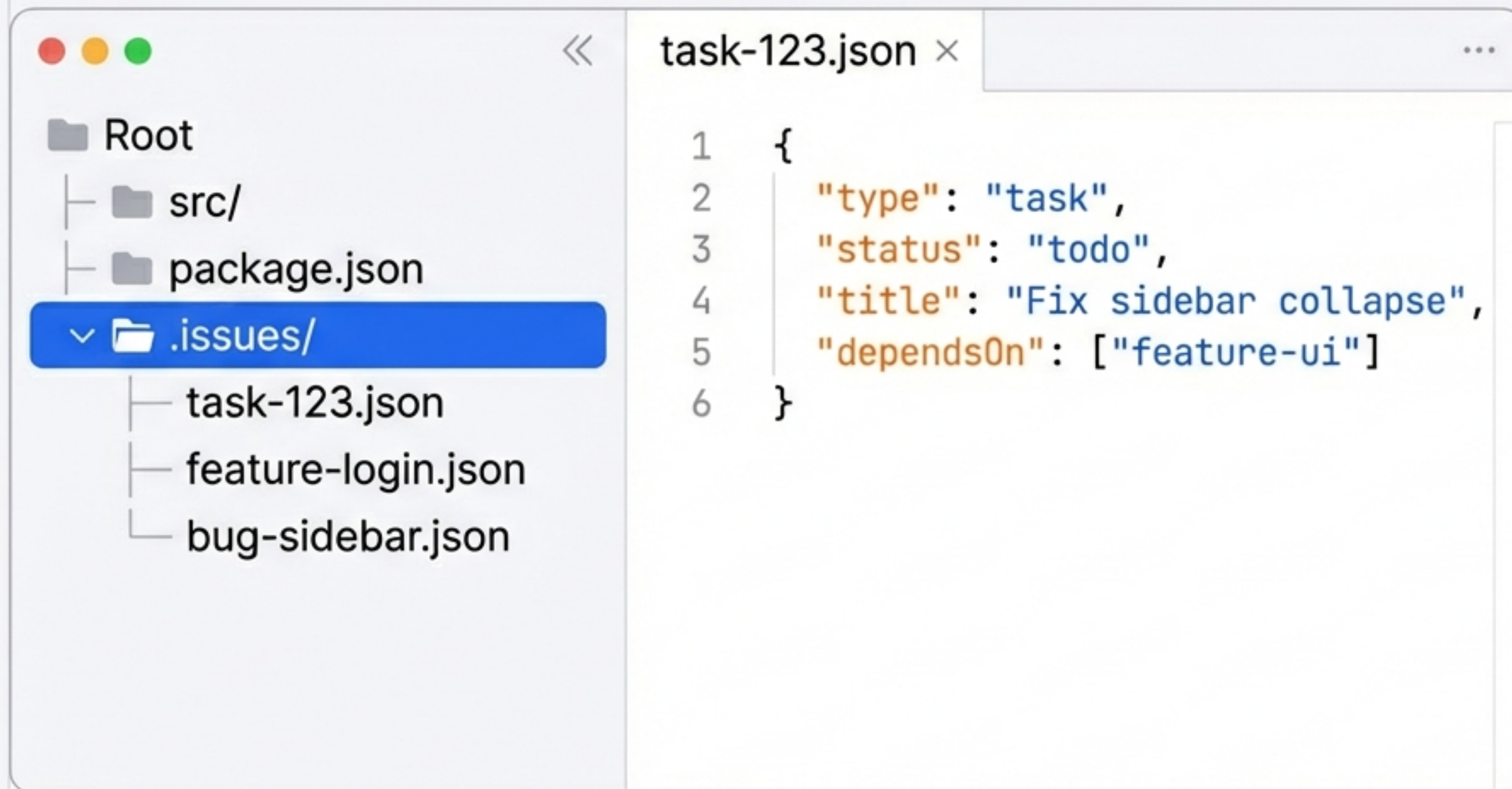
← The Context Wall →

AI Coding Agent



The Insight: Issues Are Just Files

We treat project management as code. Issues are JSON files committed directly to the repository.



Core Principles

1. File-Based Storage
2. Typed Node Graph
3. Unified Versioning (Git)

From Manual Handoffs to Semantic Integration



The Unlock: Claude Code can now understand the intent behind the project plan by reading the repository files directly.

The Multi-Claude Pattern in Inter Tight

Bidirectional Intelligence sharing a single source of truth.



v0.1.3 Scope Overview

Enhancing the Interface and Data Structure



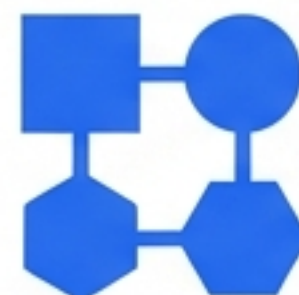
Issue Editor

A dedicated UI for creating and refining issues. Includes markdown support and structured comment threads.



Graph Visualisation

A force-directed graph view to visualise dependencies. See how Features link to Tasks and Versions.

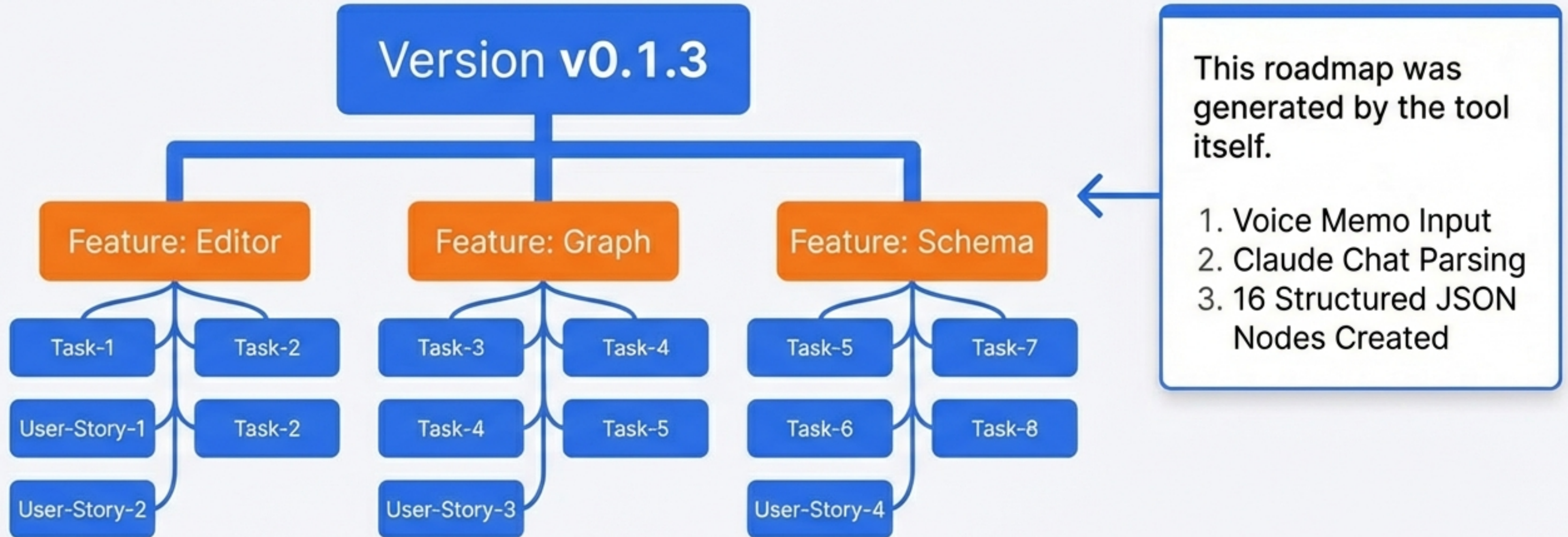


New Node Types

Expanding the schema. Introducing 'Version' and 'User-Story' types to support complex planning hierarchies.

The Source of Truth

"We are using the issue tracking system to plan and communicate its own development."

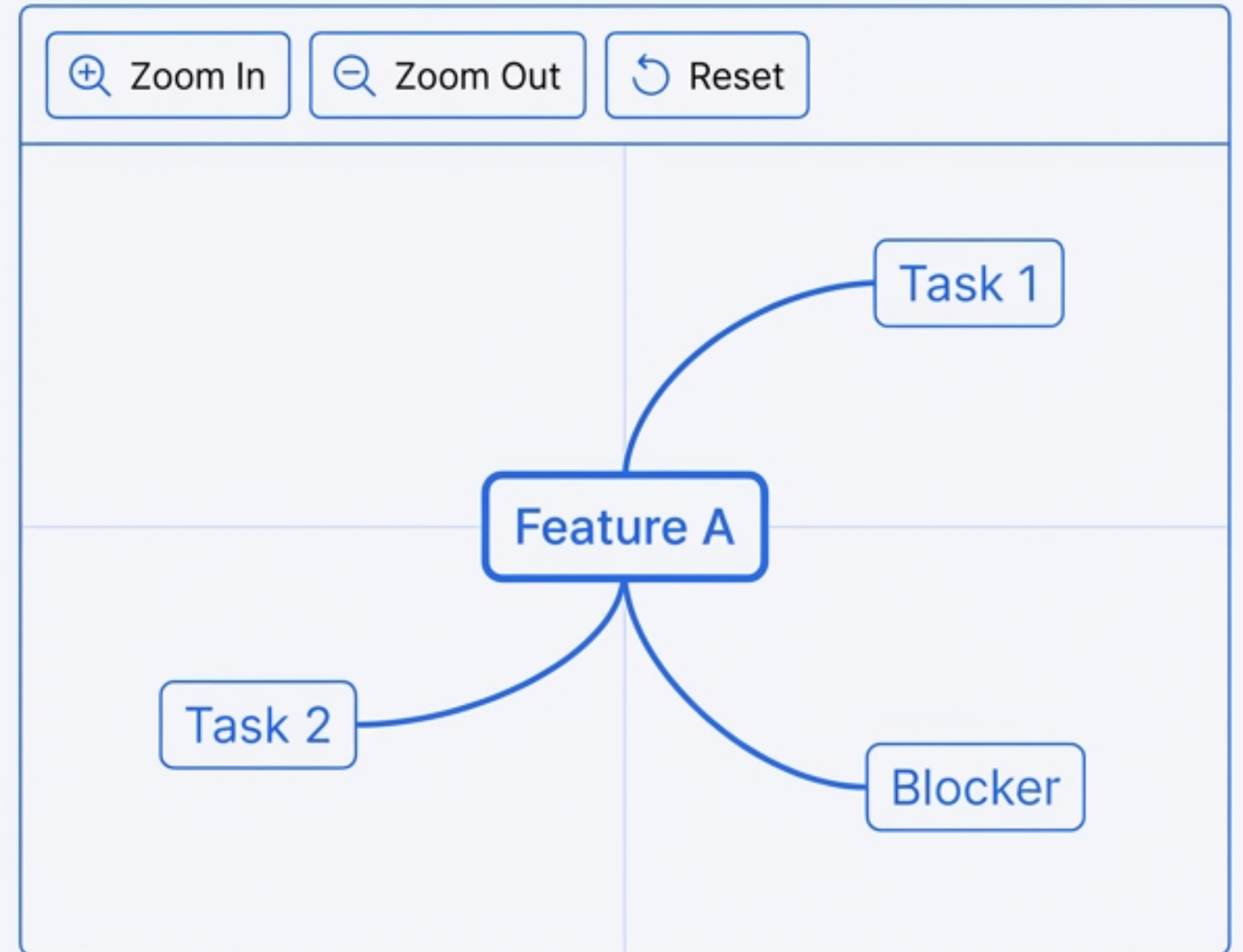


Feature Deep Dive: Graph Visualisation

Objective: Make invisible relationships visible.

Specifications

- Component: **Task-6** (D3.js force-directed graph)
- Navigation: **Task-7** ("View Graph" button on nodes)
- Backend: **Task-8** (GET /nodes/{label}/graph endpoint)



Feature Deep Dive: Issue Editor

Title

Fix Sidebar Bug

Description

Text with **Markdown** support...

Preview ☐

Properties

Status:

Todo

Type:

Bug

Comments

Add a comment...

post

Objective: Seamless editing of JSON data.

Specifications

- Core UI: **Task-2** (Edit form with PATCH wiring)
- Comments: **Task-3** (Stored in `properties.comments`)
- Markdown: **Task-4** (Preview toggle)
- Visuals: **Task-12** (Distinct colors per node type)

Task Categorisation & Analysis

UI-Only (Immediate)

Bug-1: API Docs Link

Bug-3: Sidebar Fix

Task-4: Markdown
Preview

Task-6: Graph
Component

Config Changes

Task-10: Update
node-types.json

Task-11: Update
link-types.json

Note: Support for User
Stories & Assignees

Backend Requirements

Task-8: Graph traversal
endpoint

Task-2: PATCH property
verification

Task-3: Comments
storage logic

Implementation Roadmap: Foundation

Phase 1: Config & Quick Wins

- Update `node-types.json` (version, user-story)
- Update `link-types.json`
- Fix Sidebar (Bug-3) & API Docs Link (Bug-1)

Phase 2: Editor Foundation

- Implement Markdown Preview (Task-4)
- UI Support for Node Colors (Task-12)

Future Phases



Implementation Roadmap: Execution

Phase 3: Graph Visualisation

- Build D3.js Component (Task-6)
- Implement "View Graph" Button (Task-7)
- Develop API Endpoint (Task-8)

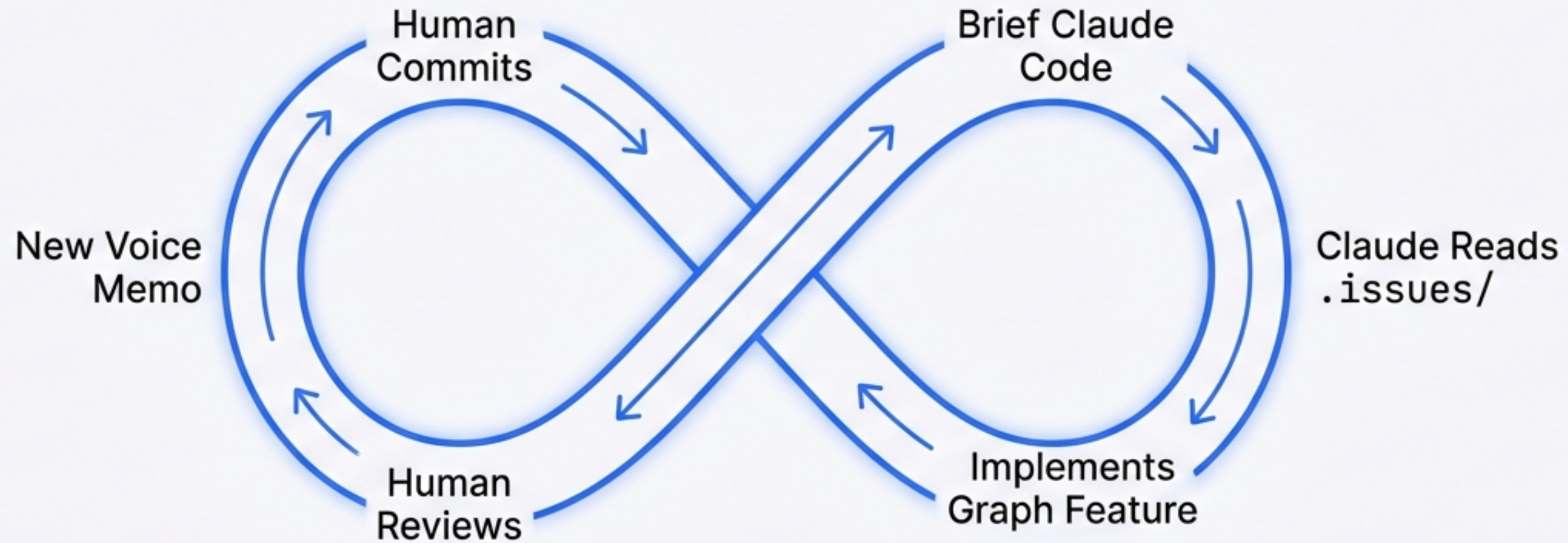
Phase 4: Full Editor

- Complete Edit Form Component (Task-2)
- Finalise Comments UI (Task-3)



Self-Hosting Development

The Meta-Loop



We are using the graph structure (Version -> Feature -> Task) to organise the work needed to build the graph visualisation feature itself.

Next Steps: Closing the Loop

Terminal Window

> ACTION ITEMS:

- [] Update node-types.json & link-types.json (Phase 1)
- [] Begin Phase 2 (Editor Foundation)
- [] Verification: Test 'New Issue' button functionality

**The system is briefed. The context is set.
Claude Code is ready to implement.**